

SWIGGY SALES ANALYSIS

SQL Analytics Query Document

Data Cleaning · Dimensional Modeling · Business Metrics · Insights

Dataset: 1,97,401 rows | Fact + Dimension Schema

PROJECT ROADMAP

End-to-end SQL analytics workflow from raw data to business insights

01

Data Ingestion

Load raw Swiggy orders
1,97,401 rows

02

Data Quality

Null checks, empty strings
& duplicate detection

03

Cleaning

Remove duplicates &
standardize records

04

Schema Design

Create dimension &
fact tables

05

ETL Load

Populate dimensions
& fact table

06

Analytics

KPIs, rankings &
business insights

01 · RAW DATA OVERVIEW

SELECT * preview of the full raw Swiggy orders dataset

SWIGGY_DATA...J DAS (52)

```
1  USE [Swiggy Database];
2
3  SELECT * FROM swiggy_data
4
5  -- Data Validation & Cleaning
6  -- Null Check
7  SELECT
8      SUM(CASE WHEN State IS NULL THEN 1 ELSE 0 END) AS null_state,
9      SUM(CASE WHEN City IS NULL THEN 1 ELSE 0 END) AS null_city,
10     SUM(CASE WHEN Order_Date IS NULL THEN 1 ELSE 0 END) AS null_order_date,
11     SUM(CASE WHEN Restaurant_Name IS NULL THEN 1 ELSE 0 END) AS null_restaurant,
12     SUM(CASE WHEN Location IS NULL THEN 1 ELSE 0 END) AS null_location,
13     SUM(CASE WHEN Category IS NULL THEN 1 ELSE 0 END) AS null_category,
14     SUM(CASE WHEN Dish_Name IS NULL THEN 1 ELSE 0 END) AS null_dish,
15     SUM(CASE WHEN Price_INR IS NULL THEN 1 ELSE 0 END) AS null_price,
16     SUM(CASE WHEN Rating IS NULL THEN 1 ELSE 0 END) AS null_rating,
17     SUM(CASE WHEN Rating_Count IS NULL THEN 1 ELSE 0 END) AS null_rating_count
18 FROM swiggy_data;
```

82 %

6

0

Ln: 3, Ch: 1 (25 chars, 1 lines) MIXED CRLF UTF-8

Results

Messages

	State	City	Order_Date	Restaurant_Name	Location	Category	Dish_Name	Price_INR	Rating	Rating
1	Karnataka	Bengaluru	2025-06-29	Anand Sweets & Savouries	Rajarajeshwari Nagar	Snack	Butter Murukku-200gm	133.899993896484	4	0
2	Karnataka	Bengaluru	2025-04-03	Srinidhi Sagar Deluxe	Kengeri	Recommended	Badam Milk	52	4.5	25
3	Karnataka	Bengaluru	2025-01-15	Srinidhi Sagar Deluxe	Kengeri	Recommended	Chow Chow Bath	117	4.69999980926514	48
4	Karnataka	Bengaluru	2025-04-17	Srinidhi Sagar Deluxe	Kengeri	Recommended	Kesari Bath	65	4.59999990463257	65
5	Karnataka	Bengaluru	2025-03-13	Srinidhi Sagar Deluxe	Kengeri	Recommended	Mix Raitha	130	4	0
6	Karnataka	Bengaluru	2025-07-08	Srinidhi Sagar Deluxe	Kengeri	Recommended	Srinidhi Sagar Special	312	4	0
7	Karnataka	Bengaluru	2025-01-21	Srinidhi Sagar Deluxe	Kengeri	Recommended	Garlic Naan	98	4	34
8	Karnataka	Bengaluru	2025-04-13	Srinidhi Sagar Deluxe	Kengeri	Recommended	Pista	137	4	0
9	Karnataka	Bengaluru	2025-05-02	Srinidhi Sagar Deluxe	Kengeri	North Indian Gravy	Panneer Butter Masala	241	4.40000009536743	29
10	Karnataka	Bengaluru	2025-07-30	Srinidhi Sagar Deluxe	Kengeri	North Indian Gravy	Dal Tadka	195	4.90000009536743	51

Query executed successfully.

localhost (17.0 RTM) | ADRIJ\ADRIJ DAS (52) | Swiggy Database | 00:00:01 | Row: 1, Col: 1 | 1,97,401 rows

02 • NULL CHECK IN DATASET

Counting NULLs in every column with SUM(CASE WHEN ...)

```
-- Data Validation & Cleaning
-- Null Check
SELECT
    SUM(CASE WHEN State IS NULL THEN 1 ELSE 0 END) AS null_state,
    SUM(CASE WHEN City IS NULL THEN 1 ELSE 0 END) AS null_city,
    SUM(CASE WHEN Order_Date IS NULL THEN 1 ELSE 0 END) AS null_order_date,
    SUM(CASE WHEN Restaurant_Name IS NULL THEN 1 ELSE 0 END) AS null_restaurant,
    SUM(CASE WHEN Location IS NULL THEN 1 ELSE 0 END) AS null_location,
    SUM(CASE WHEN Category IS NULL THEN 1 ELSE 0 END) AS null_category,
    SUM(CASE WHEN Dish_Name IS NULL THEN 1 ELSE 0 END) AS null_dish,
    SUM(CASE WHEN Price_INR IS NULL THEN 1 ELSE 0 END) AS null_price,
    SUM(CASE WHEN Rating IS NULL THEN 1 ELSE 0 END) AS null_rating,
    SUM(CASE WHEN Rating_Count IS NULL THEN 1 ELSE 0 END) AS null_rating_count
FROM swiggy_data;
```

03 • EMPTY STRING CHECK IN DATASET

WHERE clause filter to find empty strings in key columns

```
-- Blank or Empty String  
SELECT * FROM swiggy_data  
WHERE State='' OR Restaurant_Name='' OR Location='' OR Category='' OR Dish_Name='';
```

04 • DUPLICATE DETECTION IN DATASET

GROUP BY all columns + HAVING COUNT(*) > 1 to find duplicates

```
24  
25 — Duplicate Detection  
26 SELECT  
27 State, City, Order_Date, Restaurant_Name, Location, Category, Dish_Name,  
28 Price_INR, Rating, Rating_Count, COUNT(*) as CNT  
29 FROM swiggy_data  
30 GROUP BY State, City, Order_Date, Restaurant_Name, Location, Category, Dish_Name,  
31 Price_INR, Rating, Rating_Count  
32 HAVING COUNT(*) > 1;  
33
```

05 • DELETE DUPLICATE IN DATASET

CTE + ROW_NUMBER() to remove duplicate records safely

```
3  
4  — Delete Duplication  
5  WITH CTE AS ( SELECT *, ROW_NUMBER() OVER(  
6  PARTITION BY State, City, Order_Date, Restaurant_Name, Location, Category, Dish_Name,  
7  Price_INR, Rating, Rating_Count ORDER BY (SELECT NULL)) AS rn  
8  FROM swiggy_data)  
9  
10 DELETE FROM CTE WHERE rn>1;  
11
```

06 · CREATE DIMENSION TABLES

CREATE TABLE for dim_date, dim_location, restaurant & more

```
43 -- Create Schema
44 -- Date Table
45 CREATE TABLE dim_date(
46     date_id INT IDENTITY(1,1) PRIMARY KEY,
47     full_date DATE,
48     year INT,
49     month INT,
50     month_name VARCHAR(20),
51     quarter INT,
52     day INT,
53     week INT
54 )
55
56 -- dim_location
57 CREATE TABLE dim_location(
58     location_id INT IDENTITY(1,1) PRIMARY KEY,
59     State VARCHAR(100),
60     City VARCHAR(100),
61     Location VARCHAR(200)
62 );
63
64 -- dim_restaurant
65 CREATE TABLE dim_restaurant(
66     Restaurant_id INT IDENTITY(1,1) PRIMARY KEY,
67     Restaurant_Name VARCHAR(200)
68 );
69
70 -- dim_category
71 CREATE TABLE dim_category(
72     Category_id INT IDENTITY(1,1) PRIMARY KEY,
73     Category VARCHAR(200)
74 );
75
76 -- dim_dish
77 CREATE TABLE dim_dish(
78     Dish_id INT IDENTITY(1,1) PRIMARY KEY,
79     Dish_Name VARCHAR(200)
80 );
81
82
```


07 • CREATE FACT TABLE

CREATE TABLE fact_swiggy_orders with FKs to all dimension tables

```
-- Fact Table
CREATE TABLE fact_swiggy_orders(
    order_id INT IDENTITY(1,1) PRIMARY KEY,

    date_id INT,
    Price_INR DECIMAL(10,2),
    Rating DECIMAL(4,2),
    Rating_Count INT,

    location_id INT,
    Restaurant_id INT,
    Category_id INT,
    Dish_id INT,

    FOREIGN KEY(date_id) REFERENCES dim_date(date_id),
    FOREIGN KEY(location_id) REFERENCES dim_location(location_id),
    FOREIGN KEY(Restaurant_id) REFERENCES dim_restaurant(Restaurant_id),
    FOREIGN KEY(Category_id) REFERENCES dim_category(Category_id),
    FOREIGN KEY(Dish_id) REFERENCES dim_dish(Dish_id)
);
```

08 • INSERT DATA INTO DIMENSION TABLES

INSERT INTO dim tables selecting distinct values from source data

```
104 -- INSERT data in tables
105 -- dim_date
106 INSERT INTO dim_date(full_date, year, month, month_name, c
107 SELECT DISTINCT
108     Order_Date,
109     YEAR(Order_Date),
110     MONTH(Order_Date),
111     DATENAME(MONTH, Order_Date),
112     DATEPART(QUARTER, Order_Date),
113     DAY(Order_Date),
114     DATEPART(WEEK, Order_Date)
115 FROM swiggy_data
116 WHERE Order_Date IS NOT NULL;
117
118 -- dim_location
119 INSERT INTO dim_location(State, City, location)
120 SELECT DISTINCT
121     State,
122     City,
123     Location
124 FROM swiggy_data;
125
126 -- dim_restaurant
127 INSERT INTO dim_restaurant(Restaurant_Name)
128 SELECT DISTINCT
129     Restaurant_Name
130 FROM swiggy_data;
131
132 -- dim_category
133 INSERT INTO dim_category(Category)
134 SELECT DISTINCT
135     Category
136 FROM swiggy_data;
137
138 -- dim_dish
139 INSERT INTO dim_dish(Dish_Name)
140 SELECT DISTINCT
141     Dish_Name
142 FROM swiggy_data;
```

09 • INSERT DATA INTO FACT TABLE

INSERT INTO fact table joining source data to all dimension tables

```
.43
.44 -- fact_table
.45 INSERT INTO fact_swiggy_orders
.46 (
.47     date_id,
.48     Price_INR,
.49     Rating,
.50     Rating_Count,
.51     location_id,
.52     Restaurant_id,
.53     Category_id,
.54     Dish_id
.55 )
.56
.57 SELECT
.58     dd.date_id, s.Price_INR, s.Rating, s.Rating_Count,
.59     dl.location_id, dr.Restaurant_id, dc.Category_id, dsh.Dish_id
.60 FROM swiggy_data s
.61
.62 JOIN dim_date dd
.63     ON dd.full_date = s.Order_Date
.64
.65 JOIN dim_location dl
.66     ON dl.State = s.State
.67     AND dl.City = s.City
.68     AND dl.Location = s.Location
.69
.70 JOIN dim_restaurant dr
.71     ON dr.Restaurant_Name = s.Restaurant_Name
.72
.73 JOIN dim_category dc
.74     ON dc.Category = s.Category
.75
.76 JOIN dim_dish dsh
.77     ON dsh.Dish_Name = s.Dish_Name;
no
```

10 · KEY BUSINESS METRICS

Core KPI queries executed against fact_swiggy_orders

Total Orders

```
SELECT COUNT(*) AS Total_Orders
FROM fact_swiggy_orders;
```

Results		Messages
	Total_Orders	
1	197401	

Total Revenue (INR Million)

```
SELECT
FORMAT(SUM(CONVERT(FLOAT,Price_INR))/10000
00, 'N2')
+ ' INR Million' AS Total_Revenue
FROM fact_swiggy_orders;
```

Results		Messages
	Total_Revenue	
1	53.00INR Million	

Average Dish Price

```
SELECT
FORMAT(AVG(CONVERT(FLOAT,Price_INR)),
'N2')
+ ' INR' AS Avg_dish_price
FROM fact_swiggy_orders;
```

Results		Messages
	Avg_dish_price	
1	268.50INR	

Average Rating

```
SELECT AVG(Rating) AS Avg_rating
FROM fact_swiggy_orders;
```

Results		Messages
	Avg_rating	
1	4.341577	

11 · SQL QUERY RESULT

Monthly order trends: COUNT of orders by year and month

212
213
214
215
216
217
218
219
220
221
222
223

```
-- Deep-Dive Business Analysis
-- Monthly Order Trends
SELECT d.year,d.month, d.month_name,
COUNT(order_id) AS Total_Orders
FROM fact_swiggy_orders f
JOIN dim_date d
ON f.date_id = d.date_id
GROUP BY d.year,d.month, d.month_name
ORDER BY COUNT(order_id) DESC;
```

82 %
Results Messages

	year	month	month_name	Total_Orders
1	2025	1	January	25393
2	2025	8	August	25227
3	2025	5	May	25188
4	2025	7	July	24936
5	2025	4	April	24584
6	2025	3	March	24400
7	2025	6	June	24382
8	2025	2	February	23291

12 • SQL QUERY RESULT

Monthly revenue: SUM of Price_INR ranked by highest sales

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'Object Explorer' with a tree view of the 'Swiggy Database' on 'localhost (SQL Server 17.0.1000.7 - ADRIJ\ADRIJ DAS)'. The right pane shows a query window titled 'SWIGGY_DAT...J DAS (52)*' containing two SQL queries. The first query calculates the total number of orders, and the second query calculates the monthly revenue, ordered by highest sales. The results of the second query are displayed in a table below the query text.

```
216 COUNT(order_id) AS Total_Orders
217 FROM fact_swiggy_orders f
218 JOIN dim_date d
219 ON f.date_id = d.date_id
220 GROUP BY d.year, d.month, d.month_name
221 ORDER BY COUNT(order_id) DESC;
222
223
224
225
226
227 -- Monthly Revenue
228 SELECT d.year, d.month, d.month_name,
229 SUM(Price_INR) AS Total_monthly_revenue
230 FROM fact_swiggy_orders f
231 JOIN dim_date d
232 ON f.date_id = d.date_id
233 GROUP BY d.year, d.month, d.month_name
234 ORDER BY SUM(Price_INR) DESC;
235
236
```

	year	month	month_name	Total_monthly_revenue
1	2025	1	January	6823981.33
2	2025	5	May	6792621.35
3	2025	8	August	6790899.22
4	2025	7	July	6650268.51
5	2025	4	April	6590449.00
6	2025	3	March	6572738.20
7	2025	6	June	6513676.19
8	2025	2	February	6268106.67

Query executed successfully. | localhost (17.0 RTM) | ADRIJ\ADRIJ DAS (52) | Swiggy Database | 00:00:00 | Row: 1, Col: 1 | 8 rows

13 · SQL QUERY RESULT

Quarterly order trends: orders grouped by year and quarter

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'Object Explorer' with a tree view of the 'localhost (SQL Server 17.0.1000.7 - ADRIJ\ADRIJ DAS)' instance, including folders for Databases, Security, Server Objects, Replication, Always On High Availability, Management, SQL Server Agent, and XEvent Profiler.

The right pane shows a query window titled 'SWIGGY_DAT...J DAS (52)*'. The query is as follows:

```
228 SELECT d.year, d.month, d.month_name,  
229 SUM(Price_INR) AS Total_monthly_revenue  
230 FROM fact_swiggy_orders f  
231 JOIN dim_date d  
232 ON f.date_id = d.date_id  
233 GROUP BY d.year, d.month, d.month_name  
234 ORDER BY SUM(Price_INR) DESC;  
235  
236  
237  
238  
239  
240 -- Quarterly Trend  
241 SELECT d.year, d.quarter,  
242 COUNT(order_id) AS Total_Orders  
243 FROM fact_swiggy_orders f  
244 JOIN dim_date d  
245 ON f.date_id = d.date_id  
246 GROUP BY d.year, d.quarter  
247 ORDER BY COUNT(order_id) DESC;  
248
```

Below the query, the 'Results' tab is active, displaying a table with 3 rows and 4 columns: year, quarter, and Total_Orders. The data is as follows:

	year	quarter	Total_Orders
1	2025	2	74154
2	2025	1	73084
3	2025	3	50163

The status bar at the bottom indicates 'Query executed successfully.' and provides details about the connection: 'localhost (17.0 RTM) | ADRIJ\ADRIJ DAS (52) | Swiggy Database | 00:00:00 | Row: 1, Col: 1 | 3 rows'.

14 · SQL QUERY RESULT

Yearly order volume: total orders aggregated by year

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'Object Explorer' with a tree view of the 'localhost (SQL Server 17.0.1000.7 - ADRIJ\ADRIJ DAS)' server. The right pane shows a query window titled 'SWIGGY_DAT...J DAS (52)*' containing the following SQL code:

```
243 FROM fact_swiggy_orders f
244 JOIN dim_date d
245 ON f.date_id = d.date_id
246 GROUP BY d.year, d.quarter
247 ORDER BY COUNT(order_id) DESC;
248
249
250
251
252
253 -- Yearly Trend
254 SELECT d.year,
255 COUNT(order_id) AS Total_Orders
256 FROM fact_swiggy_orders f
257 JOIN dim_date d
258 ON f.date_id = d.date_id
259 GROUP BY d.year
260 ORDER BY COUNT(order_id) DESC;
261
262
263
```

Below the query window, the 'Results' tab is active, displaying a table with the following data:

	year	Total_Orders
1	2025	197401

The status bar at the bottom indicates 'Query executed successfully.' and provides details about the connection: 'localhost (17.0 RTM) | ADRIJ\ADRIJ DAS (52) | Swiggy Database | 00:00:00 | Row: 1, Col: 1 | 1 rows'.

15 · SQL QUERY RESULT

Orders by day of week: volume analysis across weekdays

The screenshot displays the SQL Server Enterprise Manager interface. The left pane shows the 'Object Explorer' with a tree view of the 'localhost (SQL Server 17.0.1000.7 - ADRIJ\ADRIJ DAS)' server. The right pane shows a query window titled 'SWIGGY_DAT...J DAS (52)*' containing two SQL queries. The first query is a simple aggregation, and the second query is a more complex one that filters for weekdays. The results pane at the bottom shows the output of the second query, which is a table with two columns: 'day_name' and 'total_orders'. The table contains seven rows of data, one for each day of the week. The status bar at the bottom indicates that the query was executed successfully.

```
255 COUNT(order_id) AS Total_Orders
256 FROM fact_swiggy_orders f
257 JOIN dim_date d
258 ON f.date_id = d.date_id
259 GROUP BY d.year
260 ORDER BY COUNT(order_id) DESC;

261
262
263
264
265
266 -- Orders by Day of Week (Mon-Sun)
267 SELECT DATENAME(WEEKDAY, d.full_date) AS day_name,
268 COUNT(order_id) AS total_orders
269 FROM fact_swiggy_orders f
270 JOIN dim_date d
271 ON f.date_id = d.date_id
272 GROUP BY DATENAME(WEEKDAY, d.full_date);
273
274
275
```

	day_name	total_orders
1	Friday	28284
2	Monday	27568
3	Saturday	28933
4	Sunday	28469
5	Thursday	28450
6	Tuesday	27413
7	Wednesday	28284

Query executed successfully. | localhost (17.0 RTM) | ADRIJ\ADRIJ DAS (52) | Swiggy Database | 00:00:00 | Row: 1, Col: 1 | 7 rows

16 · SQL QUERY RESULT

Month-over-month revenue growth using CTE and LAG

287
288
289 -- month-over-month revenue growth
290 WITH monthly_sales AS (
291 SELECT d.Year, d.month_name, SUM(f.Price_INR) AS revenue
292 FROM fact_swiggy_orders f
293 JOIN dim_date d
294 ON f.date_id = d.date_id
295 GROUP BY d.Year, d.month_name),
296 previous_month AS (
297 SELECT *, LAG(revenue) OVER (ORDER BY Year, month_name) AS previous_revenue
298 FROM monthly_sales)
299
300 SELECT Year, month_name, revenue, previous_revenue,
301 CAST((revenue - previous_revenue) * 100.0 / NULLIF(previous_revenue, 0)) AS DECIMAL(10,2)) AS mom_growth_percentage
302 FROM previous_month
303 ORDER BY Year, month_name;
304
305

82 % ⌵ ❌ 6 ⚠️ 0 ⬆️ ⬇️ Ln: 3

Results Messages

	Year	month_name	revenue	previous_revenue	mom_growth_percentage
1	2025	April	6590449.00	NULL	NULL
2	2025	August	6790899.22	6590449.00	3.04
3	2025	February	6268106.67	6790899.22	-7.70
4	2025	January	6823981.33	6268106.67	8.87
5	2025	July	6650268.51	6823981.33	-2.55
6	2025	June	6513676.19	6650268.51	-2.05
7	2025	March	6572738.20	6513676.19	0.91
8	2025	May	6792621.35	6572738.20	3.35

17 · SQL QUERY RESULT

Month generating highest revenue: ranked by SUM(Price_INR)

273
274
275
276
277
278
279
280
281
282
283
284

```
-- month generates the highest revenue
SELECT d.month_name, SUM(f.Price_INR) AS revenue
FROM fact_swiggy_orders f
JOIN dim_date d
    ON f.date_id = d.date_id
GROUP BY d.month_name
ORDER BY revenue DESC;
```

82 %
Results Messages

	month_name	revenue
1	January	6823981.33
2	May	6792621.35
3	August	6790899.22
4	July	6650268.51
5	April	6590449.00
6	March	6572738.20
7	June	6513676.19
8	February	6268106.67

18 · SQL QUERY RESULT

Top 10 cities by order volume using JOIN on dim_location

```
307
308  -- Top 10 cities by order volume
309  SELECT TOP 10
310    l.city,
311    COUNT(order_id) AS total_orders
312  FROM fact_swiggy_orders f
313  JOIN dim_location l
314  ON l.location_id = f.location_id
315  GROUP BY l.city
316  ORDER BY total_orders DESC;
317
318
319
```

82 % ▾ × 6 ⚠ 0 ↑ ↓

Results Messages

	city	total_orders
1	Bengaluru	20072
2	Mumbai	10507
3	Hyderabad	10308
4	Jaipur	10285
5	Lucknow	10192
6	New Delhi	10191
7	Ahmedabad	10175
8	Chandigarh	10060
9	Kolkata	10044
10	Chennai	10042

19 · SQL QUERY RESULT

Revenue contribution by state from fact and dim_location

```
318
319
320
321  -- Revenue Contribution by States
322  SELECT l.state,
323         SUM(f.Price_INR) AS total_revenue
324  FROM fact_swiggy_orders f
325  JOIN dim_location l
326  ON l.location_id = f.location_id
327  GROUP BY l.state
328  ORDER BY total_revenue DESC;
```

82 % 6 0

Results Messages

	state	total_revenue
1	Kamataka	5455887.73
2	Uttar Pradesh	3117359.65
3	Telangana	3021656.62
4	Maharashtra	3015573.35
5	Delhi	2829180.60
6	Gujarat	2815536.27
7	Punjab	2804991.82
8	West Bengal	2662213.76
9	Tamil Nadu	2642594.63
10	Raiasthan	2502833.61

20 • SQL QUERY RESULT

Cities with highest average restaurant ratings

```
331
332
333  -- cities have the highest average-rated restaurants
334  SELECT
335      l.City,
336      AVG(f.Rating) AS avg_rating,
337      COUNT(*) AS total_orders
338  FROM fact_swiggy_orders f
339  JOIN dim_location l
340      ON f.location_id = l.location_id
341  GROUP BY l.City
342  HAVING COUNT(*) >= 100
343  ORDER BY avg_rating DESC;
344
345
346
```

2 % 6 0

	City	avg_rating	total_orders
1	Kochi	4.444004	5029
2	Kolkata	4.410921	10044
3	Aizawl	4.409715	3088
4	Panaji	4.385367	5030
5	Ahmedabad	4.385189	10175
6	Guwahati	4.375655	5032
7	Lucknow	4.374558	10192
8	Chennai	4.365166	10042
9	Bhubaneswar	4.357692	5018
10	Gurgaon	4.345294	5005

21 · SQL QUERY RESULT

Percentage of total revenue contributed by each state

```
348
349 -- percentage of total revenue comes from each state
350 WITH state_revenue AS
351 (SELECT l.State, SUM(f.Price_INR) AS revenue
352  FROM fact_swiggy_orders AS f
353  JOIN dim_location AS l
354   ON f.location_id = l.location_id
355  GROUP BY l.State)
356
357 SELECT State, revenue AS state_revenue,
358        CONVERT(DECIMAL(10,2), revenue * 100.0 / NULLIF(SUM(revenue) OVER (),0)) AS revenue_percentage
359 FROM state_revenue
360 ORDER BY revenue_percentage DESC;
```

82 %

6 0



Results

Messages

	State	state_revenue	revenue_percentage
1	Kamataka	5455887.73	10.29
2	Uttar Pradesh	3117359.65	5.88
3	Telangana	3021656.62	5.70
4	Maharashtra	3015573.35	5.69
5	Delhi	2829180.60	5.34
6	Gujarat	2815536.27	5.31
7	Punjab	2804991.82	5.29
8	West Bengal	2662213.76	5.02
9	Tamil Nadu	2642594.63	4.99
10	Raiasthan	2502833.61	4.72

22 • SQL QUERY RESULT

Prioritize restaurants by orders, rating, price & revenue

```
365
366 -- restaurants should Swiggy prioritize for promotion
367 SELECT
368     r.Restaurant_Name,
369     COUNT(*) AS total_orders,
370     AVG(f.Rating) AS avg_rating,
371     AVG(f.Price_INR) AS avg_price,
372     SUM(f.Price_INR) AS revenue
373 FROM fact_swiggy_orders f
374 JOIN dim_restaurant r
375     ON f.restaurant_id = r.restaurant_id
376 GROUP BY r.Restaurant_Name
377 HAVING AVG(f.Rating) >= 4.0
378     AND COUNT(*) >= 100
379     AND AVG(f.Price_INR) BETWEEN 150 AND 400
380 ORDER BY revenue DESC;
381
382
```

82 % 6 0 ↑ ↓

Results Messages

	Restaurant_Name	total_orders	avg_rating	avg_price	revenue
1	KFC	12957	4.303565	327.657774	4245461.78
2	McDonald's	13528	4.395106	247.076846	3342455.58
3	Pizza Hut	6529	4.198192	326.736971	2133265.69
4	Burger King	7115	4.340744	267.114278	1900518.09
5	Domino's Pizza	5489	4.372690	333.937934	1832985.32
6	Olio - The Wood Fired Pizzeria	3239	4.294411	380.589996	1232731.00
7	LunchBox - Meals and Thalys	4700	4.367744	234.285319	1101141.00
8	Baskin Robbins - Ice Cream Desserts	4197	4.426042	205.049306	860591.94
9	Faasos - Wraps, Rolls & Shawarma	3256	4.364281	239.623771	780215.00
10	The Good Bowl	2665	4.344878	252.661538	673343.00

✓ Query executed successfully. localhost (11

23 · SQL QUERY RESULT

High-value restaurants: total orders, revenue and avg rating

```
383
384
385
386  -- Identify high-value restaurants
387  SELECT
388      r.Restaurant_Name,
389      COUNT(*) AS total_orders,
390      SUM(f.Price_INR) AS total_revenue,
391      AVG(f.Rating) AS avg_rating
392  FROM fact_swiggy_orders f
393  JOIN dim_restaurant r
394      ON f.Restaurant_id = r.Restaurant_id
395  GROUP BY r.Restaurant_Name
396  HAVING SUM(f.Price_INR) > 100000
397      AND AVG(f.Rating) >= 4
398  ORDER BY total_revenue DESC;
399
400
```

82 %

6

0

↑

↓

Results

Messages

	Restaurant_Name	total_orders	total_revenue	avg_rating
1	KFC	12957	4245461.78	4.303565
2	McDonald's	13528	3342455.58	4.395106
3	Pizza Hut	6529	2133265.69	4.198192
4	Burger King	7115	1900518.09	4.340744
5	Domino's Pizza	5489	1832985.32	4.372690
6	Olio - The Wood Fired Pizzeria	3239	1232731.00	4.294411
7	LunchBox - Meals and Thalys	4700	1101141.00	4.367744
8	Baskin Robbins - Ice Cream Desserts	4197	860591.94	4.426042
9	Faasos - Wraps, Rolls & Shawarma	3256	780215.00	4.364281
10	The Good Bowl	2665	673343.00	4.344878

Query executed successfully

24 · SQL QUERY RESULT

Top restaurants ranked by total revenue generated

```
404
405 -- Top 10 Restaurants by orders
406 SELECT TOP 10
407   r.restaurant_name,
408   SUM(f.price_INR) AS Total_revenue
409 FROM fact_swiggy_orders f
410 JOIN dim_restaurant r
411   ON r.restaurant_id = f.restaurant_id
412  GROUP BY r.restaurant_name
413  ORDER BY Total_revenue DESC;
414
415
416
```

82 %

✖ 6 ⚠ 0

↑ ↓

Results

Messages

	restaurant_name	Total_revenue
1	KFC	4245461.78
2	McDonald's	3342455.58
3	Pizza Hut	2133265.69
4	Burger King	1900518.09
5	Domino's Pizza	1832985.32
6	Olio - The Wood Fired Pizzeria	1232731.00
7	LunchBox - Meals and Thalys	1101141.00
8	Baskin Robbins - Ice Cream Desserts	860591.94
9	Faasos - Wraps, Rolls & Shawarma	780215.00

25 · SQL QUERY RESULT

Top food categories ranked by order volume

```
416
417
418  -- Top Categories by Order Volume
419  SELECT c.category, COUNT(order_id) AS total_orders
420  FROM fact_swiggy_orders f
421  JOIN dim_category c
422  ON f.Category_id = c.Category_id
423  GROUP BY c.Category
424  ORDER BY total_orders DESC;
425
```

82 %

✖ 6 ⚠ 0

↑ ↓

Results

Messages

	category	total_orders
1	Recommended	24097
2	Desserts	3582
3	Main Course	2983
4	Beverages	2682
5	BURGERS	2538
6	Sweets	1954
7	McSaver Combos (2 Pc Meals)	1884
8	Exclusive Deals (Save upto 40%)	1717
9	Starters	1692

26 · SQL QUERY RESULT

Most ordered dishes across the entire platform

```
426
427
428  -- Most Ordered Dishes
429  SELECT d.dish_name, COUNT(order_id) AS order_count
430  FROM fact_swiggy_orders f
431  JOIN dim_dish d
432  ON d.Dish_id = f.Dish_id
433  GROUP BY d.Dish_Name
434  ORDER BY order_count DESC;
435
436
437
```

82 % ▾ × 6 ⚠ 0 ↑ ↓

Results		Messages
	dish_name	order_count
1	Veg Fried Rice	321
2	Choco Lava Cake	303
3	Jeera Rice	265
4	Paneer Butter Masala	262
5	French Fries	248
6	Chicken Sausage	230
7	Chicken Fried Rice	228
8	Butter Naan	218
9	Margherita Pizza	203

27 · SQL QUERY RESULT

Cuisine performance: order volume + average rating by category

```
438
439  -- Cuisine Performance(Orders + Avg Rating)
440  SELECT c.category, COUNT(order_id) AS total_orders,
441         AVG(f.rating) AS avg_rating
442  FROM fact_swiggy_orders f
443  JOIN dim_category c
444  ON f.Category_id = c.Category_id
445  GROUP BY c.Category
446  ORDER BY total_orders DESC;|
447
448
449
```

82 %

✖ 6

⚠ 0

↑

↓

Results

Messages

	category	total_orders	avg_rating
1	Recommended	24097	4.321782
2	Desserts	3582	4.371635
3	Main Course	2983	4.310191
4	Beverages	2682	4.368717
5	BURGERS	2538	4.324940
6	Sweets	1954	4.455527
7	McSaver Combos (2 Pc Meals)	1884	4.413800
8	Exclusive Deals (Save upto 40%)	1717	4.348806
9	Starters	1692	4.304846

28 · SQL QUERY RESULT

Restaurants that contribute the highest total revenue

```
449
450
451 -- restaurants contribute the most revenue
452 SELECT
453     r.Restaurant_Name,
454     COUNT(*) AS total_orders,
455     SUM(f.Price_INR) AS revenue
456 FROM fact_swiggy_orders f
457 JOIN dim_restaurant r
458     ON f.Restaurant_id = r.Restaurant_id
459 GROUP BY r.Restaurant_Name
460 ORDER BY revenue DESC;
461
```

82 % 6 0

Results Messages

	Restaurant_Name	total_orders	revenue
1	KFC	12957	4245461.78
2	McDonald's	13528	3342455.58
3	Pizza Hut	6529	2133265.69
4	Burger King	7115	1900518.09
5	Domino's Pizza	5489	1832985.32
6	Olio - The Wood Fired Pizzeria	3239	1232731.00
7	LunchBox - Meals and Thalís	4700	1101141.00
8	Baskin Robbins - Ice Cream Desserts	4197	860591.94
9	Faasos - Wraps, Rolls & Shawarma	3256	780215.00

29 · SQL QUERY RESULT

State-wise revenue share as percentage of total sales

```
464
465
466  -- percentage of total revenue comes from each state
467  WITH state_revenue AS (SELECT l.State, SUM(f.Price_INR) AS revenue
468      FROM fact_swiggy_orders f
469      JOIN dim_location l
470      ON f.location_id = l.location_id
471      GROUP BY l.State
472  )
473
474  SELECT State, revenue, revenue * 100.0 / SUM(revenue) OVER () AS revenue_percentage
475  FROM state_revenue
476  ORDER BY revenue DESC;|
477
478
479
```

82 %  6  0  

 Results  Messages

	State	revenue	revenue_percentage
1	Karnataka	5455887.73	10.293595
2	Uttar Pradesh	3117359.65	5.881506
3	Telangana	3021656.62	5.700944
4	Maharashtra	3015573.35	5.689466
5	Delhi	2829180.60	5.337800
6	Gujarat	2815536.27	5.312057
7	Punjab	2804991.82	5.292163
8	West Bengal	2662213.76	5.022785
9	Tamil Nadu	2642594.63	4.985769

30 · SQL QUERY RESULT

High-rated restaurants with low order volume

```
480
481
482 -- restaurants have high ratings but low sales
483 SELECT
484     r.Restaurant_Name,
485     COUNT(*) AS total_orders,
486     AVG(f.Rating) AS avg_rating
487 FROM fact_swiggy_orders f
488 JOIN dim_restaurant r
489     ON f.restaurant_id = r.Restaurant_id
490 GROUP BY r.Restaurant_Name
491 HAVING AVG(f.Rating) >= 4.0
492     AND COUNT(*) < 100
493 ORDER BY avg_rating DESC;
494
```

82 %

6

0

Results

Messages

	Restaurant_Name	total_orders	avg_rating
1	Sakana	95	4.815789
2	Vijay Dairy	52	4.750000
3	Jagannath Mandir Ama Prasad	8	4.750000
4	Radhey Lal's Parampara Sweets	84	4.736904
5	I Deli Cafe	47	4.736170
6	Yadav Doodh Dairy	70	4.728571
7	Perambur Sri Srinivasa Sweets & Snacks	20	4.705000
8	Wooden Plate	53	4.692452
9	Natural Ice Cream	87	4.683908

31 · SQL QUERY RESULT

Order volume distribution across different price ranges

```
499
500
501  -- Total Orders by Price Range
502  SELECT
503      CASE
504          WHEN price_INR < 100 THEN 'Under 100'
505          WHEN price_INR BETWEEN 100 AND 199 THEN '100-199'
506          WHEN price_INR BETWEEN 200 AND 299 THEN '200-299'
507          WHEN price_INR BETWEEN 300 AND 499 THEN '300-499'
508          ELSE '500+'
509      END AS price_range,
510      COUNT(order_id) AS total_orders
511  FROM fact_swiggy_orders
512  GROUP BY
513      CASE
514          WHEN price_INR < 100 THEN 'Under 100'
515          WHEN price_INR BETWEEN 100 AND 199 THEN '100-199'
516          WHEN price_INR BETWEEN 200 AND 299 THEN '200-299'
517          WHEN price_INR BETWEEN 300 AND 499 THEN '300-499'
518          ELSE '500+'
519      END
520  ORDER BY total_orders DESC;
521
522
```

82 %

Results Messages

	price_range	total_orders
1	100-199	56189
2	200-299	54567
3	300-499	43758
4	Under 100	26795
5	500+	16092

32 · SQL QUERY RESULT

Customer rating distribution: count of orders per rating value

523
524
525 -- Rating Count
526 SELECT rating, COUNT(*) AS rating_count
527 FROM fact_swiggy_orders
528 GROUP BY rating
529 ORDER BY rating_count DESC;
530
531
532
533

82 %
6
0
↑ ↓

ResultsMessages

	rating	rating_count
1	4.40	85642
2	4.30	13698
3	4.60	10840
4	4.50	9946
5	5.00	9401
6	4.70	9089
7	4.80	8809
8	4.20	8214
9	4.10	7619

PROJECT SUMMARY

- ▶ Loaded and profiled 1,97,401 raw Swiggy order records
- ▶ Performed comprehensive data quality checks (nulls, empty strings, duplicates)
- ▶ Cleaned the dataset by removing duplicate entries
- ▶ Designed and created dimension tables + fact_swiggy_orders
- ▶ Executed ETL inserts into the dimensional model
- ▶ Derived core KPIs: Total Orders, Revenue, Avg Price, Avg Rating
- ▶ Produced extensive analytical queries for business insights